ICT162

Timed Online Assignment – January Semester 2021

Object Oriented Programming

Thursday, 27 May 2021 **1:00 pm – 3:30 pm**

Time allowed: 2.5 hours

INSTRUCTIONS TO STUDENTS:

1. This Timed Online Assignment (TOA) contains **FOUR (4)** questions and comprises **SEVENTEEN (17)** pages (including cover page).

2. You must answer **ALL** questions.

3. If you have any queries about a question, or believe there is an error in the question, briefly explain your understanding and assumptions about that question before attempting it.

4. You MUST **submit your answers via Canvas (similar to TMA submission) at the end time of this TOA** (as stated on this cover page). The 15 minutes grace period as shown on Canvas is strictly meant for technical issues encountered during submission. Thereafter, you will not be able to submit your answers and you will be considered as having withdrawn from the course. **No appeal will be allowed.**

5. You are to include the following particulars in your submission: TOA Course Code, Your Name, and SUSS PI No. Name your submission file as TOACourseCode_FullName_StudentPI. Use underscore and NOT Space. *Example: TOAICT162_RaphaelLee_T1923161 (omit D/O, S/O).*

6. Your submission should consist of only one file and **must not exceed 500MB** in size. The file must be a Microsoft Word file saved in .doc or .docx format. All answers are to be typed. Flowcharts and graphs may be scanned or photographed and embedded in the Word file provided it does not exceed the file size limit of 500MB. Images of handwritten answers will not be marked.

7. To prevent plagiarism and collusion, your submission will be reviewed by Turnitin. The Turnitin report will only be made available to the marker and you will not be able to view it.

8. The University takes plagiarism and collusion seriously, and your Turnitin report will be examined thoroughly as part of the marking process.

Answer all questions. (Total 100 marks)

Question 1

Study the class diagram in Figure Q1. The Driver and Trip classes are described in Table Q1(a) and Table Q1(b) respectively.

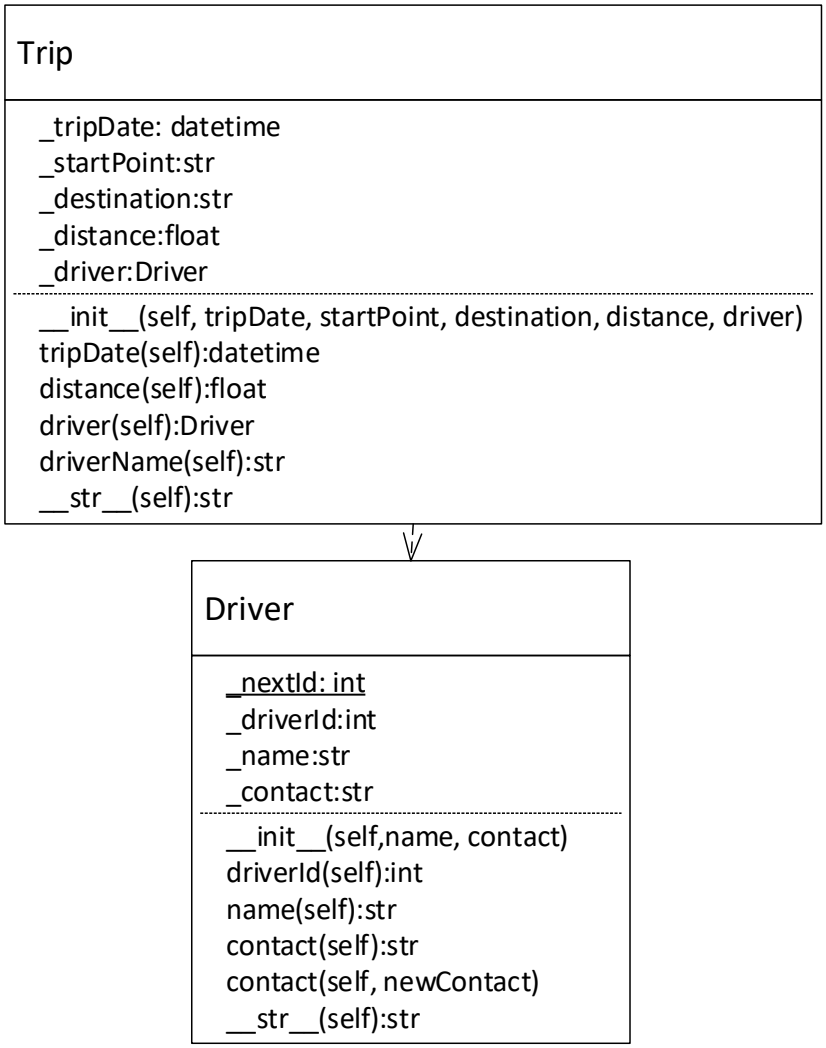

Figure Q1

The Driver class has the following attributes: • _nextId is used to auto generate running numbers for the driver id of drivers, starting from 1. • _driverId uniquely identifies a driver. • _name records the name of a driver. • _contact records the contact of a driver.
The Driver class has the following methods: • __init__(self, name, contact) initialises a new driver with an auto-generated driver id, and the parameters name and contact. • Getter methods for driverId, name and contact. • Setter method for contact. • __str__(self) returns a string representation of a driver. An example string is shown here: `Driver Id: 1 Name John Contact: 98989898`

Table Q1(a)

The Trip class has the following attributes: • _tripDate is the date and time of the trip. • _startPoint is the starting point of the trip. • _destination of the trip. • _distance is the distance between the starting point and destination of trip.
The Trip class has the following methods: • __init__(self, tripDate, startPoint, destination, distance, driver) initialises a new trip with the parameters tripDate, startPoint, destination, distance and driver. • Getter methods for tripDate, distance, driver and driver name. • __str__(self) returns a string representation of a trip, including the details of the driver. An example string is shown here: `30 May 2020, 17:45 From: Tampines Mall To: Parkway Parade Distance: 11.2km Driver Id: 1 Name John Contact: 98989898`

Table Q1(b)

(a) Differentiate between object composition and inheritance and then identify and explain the relationship between Driver and Trip. If you identified the relationship as object composition, explain why the relationship should not be inheritance. Otherwise, explain why the relationship should not be object composition.

(5 marks)

(b) Write a Python class definition for the Driver class.

(5 marks)

(c) Write a Python class definition for the Trip class.

(5 marks)

Question 2

GoWhere is a private car ride company that offers two types of travel cards to customers: 1) trip travel card where a customer gets one long (more than 12 km) trip and 3 short trips, and 2) distance travel card that gives customers as many trips as long as the accumulated distance is not more than 50 km. Travel cards have expiry dates and can be topped up as long as they have not expired.

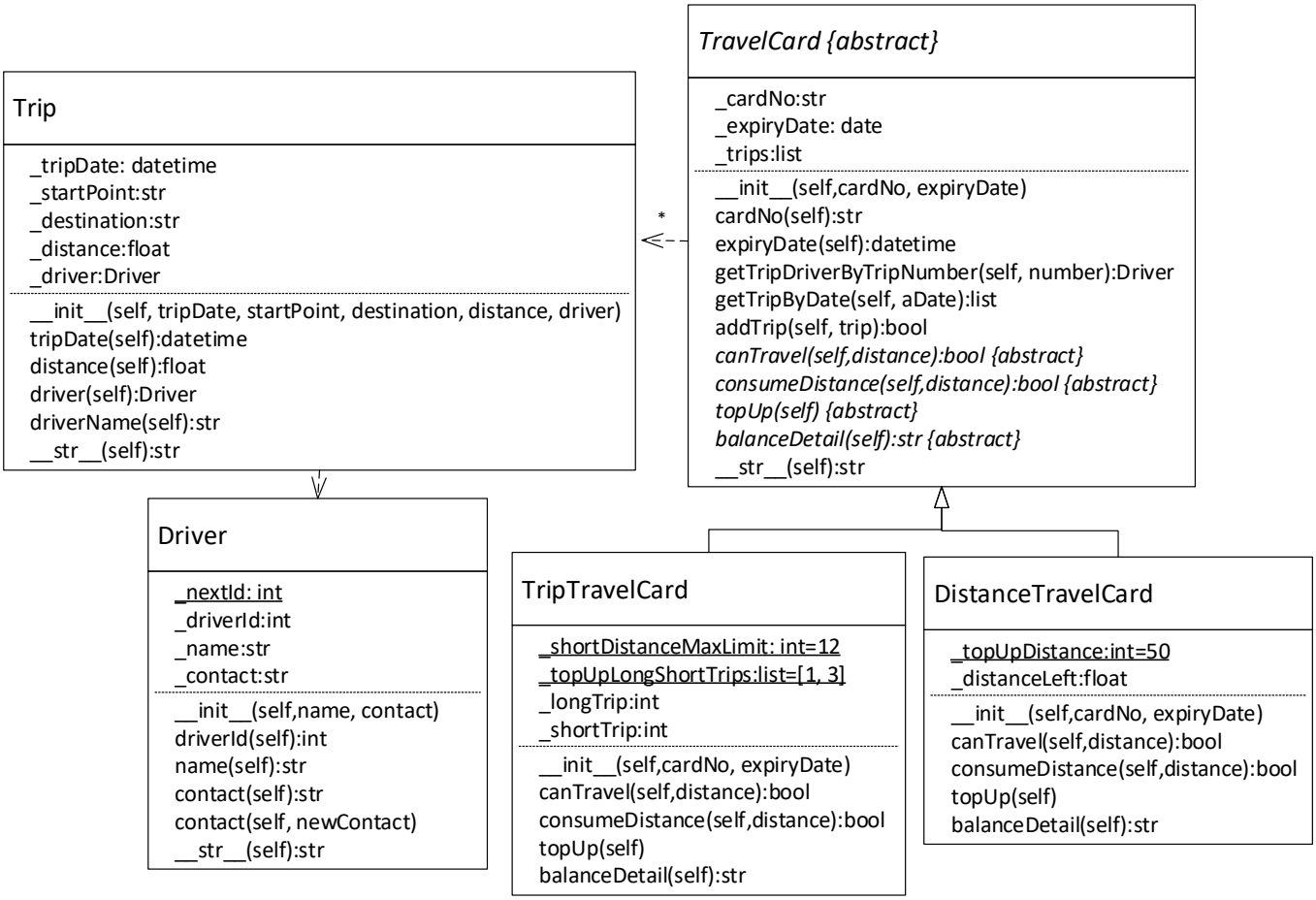

Figure Q2

Study the class diagram in Figure Q2. The TravelCard class, the TripTravelCard class, and the DistanceTravelCard class are described in Table Q2(a) to Table Q2(c) respectively. You will implement only the TravelCard and TripTravelCard classes.

The TravelCard class is an abstract class. It has the following attributes:

- _cardNo records the card number
- _expiryDate records the expiry date of the travel card
- _trips records a list of trips taken using the travel card

The TravelCard class has the following methods:

- __init__(self, cardNo, expiryDate) initialises a new travel card with the parameters cardNo and expiryDate, and set _trips as an empty list.
- Getter methods for cardNo and expiryDate.
- getTripDriverByTripNumber(self, n):Driver returns the driver of the n-th trip. If n is 1, the driver of the first trip in the list is returned, if n is 2, the driver of the second trip is returned, and so on. Return None if n is zero or negative or if the n-th trip does not exist.
- getTripByDate(self, aDate):list returns a list of trips taken on aDate. The method returns None if no trip is taken on aDate.
- addTrip(self, trip):bool returns True if the trip can be added to _trips. A trip can be added under the condition that
  - the expiry date of the travel card is on or after the date of the trip, and
  - the travel card has sufficient balance to make the trip, as determined by the method canTravel(self, distance).

  If the condition is satisfied, the trip can be added to _trips, and the method deducts from the balance of the card by calling the method consumeDistance(self, distance).
- An abstract method canTravel(self, distance):bool.
- An abstract method consumeDistance(self, distance):bool
- An abstract method topUp(self).
- An abstract method balanceDetail(self):str
- __str__(self) returns a string representation of a card, including the balance details of the card(shown in bold, dependent on the type of travel card) and its trips. Three example strings are shown here:

```
Card Number:C123 Expiry Date: 30 Jun 2021 Remaining Long trips: 0 Short
trips: 1
30 May 2020, 12:35 From: Parkway Parade  To: Tampines Mall   Distance:
11.0km Driver Id: 2 Name Tom Contact: 88998899
30 May 2020, 17:45 From: Tampines Mall   To: Parkway Parade  Distance:
11.2km Driver Id: 1 Name John Contact: 98989898
04 Jun 2020, 12:35 From: Jurong Point    To: Parkway Parade  Distance:
20.3km Driver Id: 2 Name Tom Contact: 88998899

Card Number:C003 Expiry Date: 01 Jan 2020 Remaining Long trips: 1 Short
trips: 3
No trips yet

Card Number:D001 Expiry Date: 01 Jan 2022 Remaining Distance: 29.7km
05 Jun 2020, 08:30 From: Jurong Point    To: Parkway Parade  Distance:
20.3km Driver Id: 2 Name Tom Contact: 88998899
```

Table Q2(a)

The TripTravelCard class is a subclass of the TravelCard class. It has the following attributes in addition to the attributes of the TravelCard class: • _shortDistanceMaxLimit is12 km or the distance a short trip cannot exceed. Any trip with a distance greater than 12 km is a long trip. • _topUpLongShortTrips is a list of 2 items, 1 and 3, the number of the long trips and short trips respectively, that a trip travel card has when initialised. The numbers in _topUpLongShortTrips are also used to add to the long trips and short trips when a trip travel card is topped up. • _longTrip records the number of long trips remaining for a trip travel card. _longTrip is initially set to the first item in _topUpLongShortTrips. • _shortTrip records the number of short trips remaining for a trip travel card. _shortTrip is initially set to the second item in _topUpLongShortTrips.
The TripTravelCard class has the following methods, in addition to the methods of the TravelCard class: • __init__(self, cardNo, expiryDate) which also initialises _longTrip and _shortTrip with the items in the list, _topUpLongShortTrips. • canTravel(self, distance):bool returns True if the card has sufficient balance for the distance, that is, – the distance is short and _shortTrip is positive. – the distance is short but although _shortTrip is zero, _longTrip is positive. One long trip can be converted to two short trips, one of which can be used for the short distance. – the distance is long and _longTrip is positive. – the distance is long but although _longTrip is zero, _shortTrip is at least 3. Three short trips can be converted to one long trip which can then be used for the long distance. The method returns False if the trip travel card does not have enough long and short trips for the distance. • consumeDistance(self, distance):bool returns True when trip travel card can be used for the distance, and the card balance is successfully deducted, with remaining trips converted from long to short or vice versa, if necessary. • topUp(self) adds to long and short trips, according to the numbers in the list,_topUpLongShortTrips. • balanceDetail(self):str returns a string representation for the remaining trips. An example string is shown here: `Long trips: 1 Short trips: 3`

Table Q2(b)

The DistanceTravelCard class is a subclass of the TravelCard class. It has the following attributes in addition to the attributes of the TravelCard class: • _topUpDistance is 50 km which is the distance left a distance travel card has when initialised, and amount to add to the distance left when a distance travel card is topped up. • _distanceLeft records the remaining distance that a distance travel card has.
The DistanceTravelCard class has the following methods, in addition to the methods of the TravelCard class: • __init__(self,cardNo, expiryDate) which also initialises _distanceLeft with_topUpDistance. • canTravel(self,distance):bool returns True if the distance left in the card is at least the parameter distance. The method returns False otherwise. • consumeDistance(self, distance):bool returns True when trip travel card can be used for the distance, and the card balance is successfully deducted. • topUp(self) adds _topUpDistance to the distance left in the distance travel card. • balanceDetail(self):str returns a string representation for the remaining distance. An example string is shown here: `Distance: 29.7km`

Table Q2(c)

Demonstrate your understanding of how class hierarchy and association are used to organize information, and then construct the class hierarchy and association according to specification.

(a) Define the TravelCard class.

(15 marks)

(b) Define the TripTravelCard class.

(15 marks)

Question 3

Apply the principles of object-oriented programming and develop an application where travel cards for customers are managed. Study the class diagram in Figure Q3.

The CustomerCardException class and the Customer class are described in Table Q3(a) and Table Q3(b) respectively.

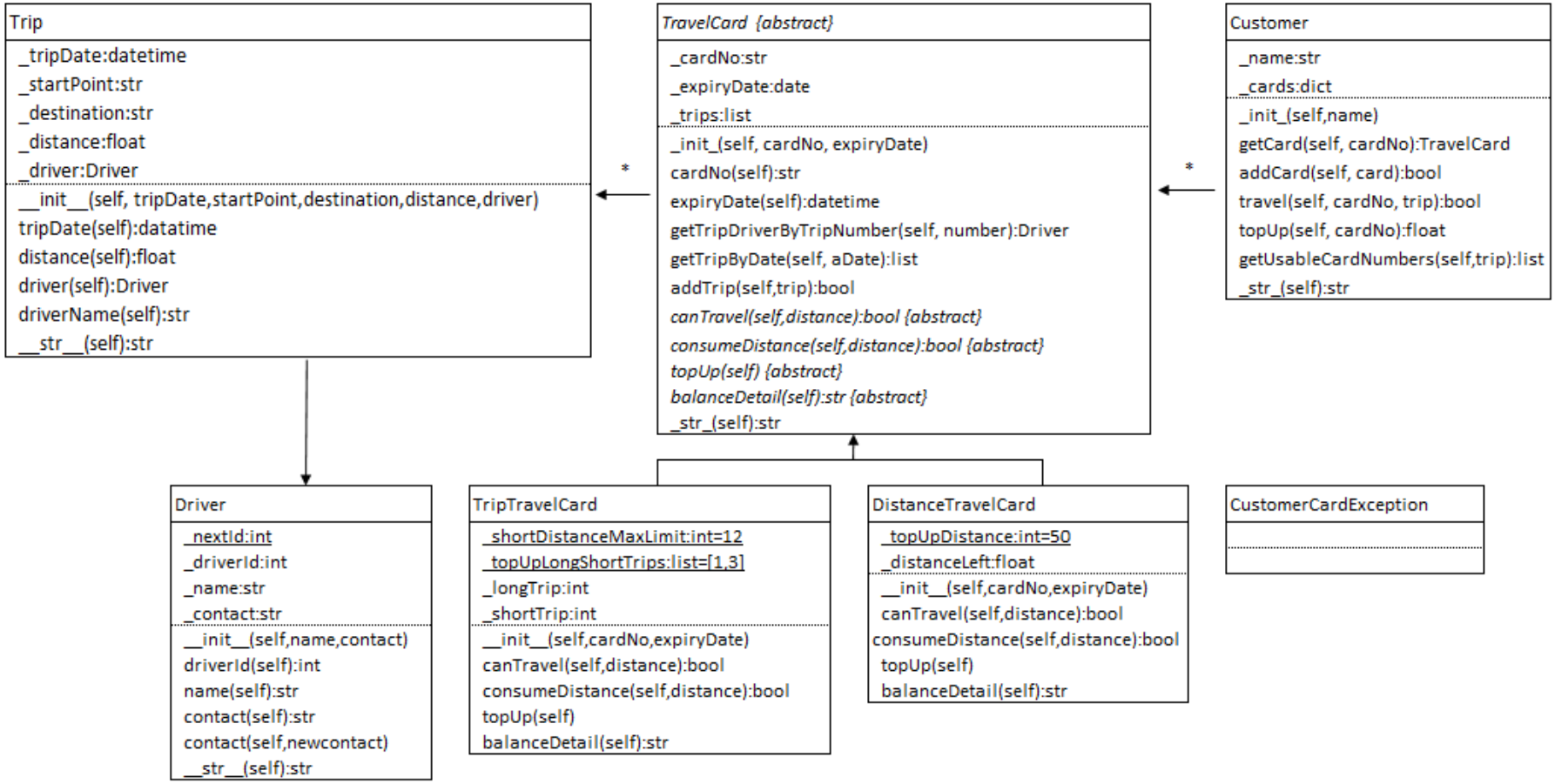

Figure Q3

ICT162 Copyright © 2021 Singapore University of Social Sciences (SUSS)

The CustomerCardException class is a subclass of Exception. It has no additional attribute. A CustomerCardException object is raised when there is a business rule violation.
The CustomerCardException class has no additional method.

Table Q3(a)

The Customer class has the following attributes: • _name of customer. • _cards is a dictionary of travel cards owned by the customer with card number as key and card as value.

The Customer class has the following methods:
• __init__(self, name) initialises a new customer with the parameter name. A customer starts with an empty list of cards.
• getCard(self, cardNo):TravelCard returns a travel card with the parameter cardNo if _cards contain a card with the parameter cardNo. Otherwise, it returns None.
• addCard(self, card):bool adds the parameter card to _cards, if _card does not contain the card already. If the add is successful, the method returns True. Otherwise, the method raises a CustomerCardException exception with the message Duplicate card number. Card cannot be added.
• travel(self, cardNo, trip):bool adds the parameter trip to the card with the parameter cardNo.
– If the add is successful, the method returns True.
– If the add fails, it returns False.
– If a card with cardNo cannot be located, the method raises a CustomerCardException exception with the message Cannot locate card for travel.
• topUp(self, cardNo):bool tops up a card with the parameter cardNo.
– If the top up is successful, the method returns True.
– If a card with cardNo cannot be located, the method raises a CustomerCardException exception with the message Cannot locate card for top up.
– If the card has expired on today's date, the method raises a CustomerCardException exception with the message Card has expired. Cannot top up.
• getUsableCardNumbers(self, trip):list returns a list of card number of cards that have sufficient balance for the parameter trip. If the list is empty, the method raises a CustomerCardException exception with the message No card can be used for trip. Buy a new card or top up a card first
• __str__(self):str returns a __str__(self) returns a string representation of a customer. Two example strings are shown here:
`Name: Peter`
`No card yet`
 	
`Name: Peter`
`Card Number:C123 Expiry Date: 30 Jun 2021 Remaining Long trips: 0 Short trips: 1`
`30 May 2020, 12:35 From: Parkway Parade To: Tampines Mall Distance: 11.0km Driver Id: 2 Name Tom Contact: 88998899`
`30 May 2020, 17:45 From: Tampines Mall To: Parkway Parade Distance: 11.2km Driver Id: 1 Name John Contact: 98989898`
`04 Jun 2020, 12:35 From: Jurong Point To: Parkway Parade Distance: 20.3km Driver Id: 2 Name Tom Contact: 88998899`

```
Card Number:C003 Expiry Date: 01 Jan 2020 Remaining Long trips: 1 Short
trips: 3
No trips yet

Card Number:D001 Expiry Date: 01 Jan 2022 Remaining Distance: 29.7km
05 Jun 2020, 08:30 From: Jurong Point    To: Parkway Parade  Distance:
20.3km Driver Id: 2 Name Tom Contact: 88998899
```

Table Q3(b)

(a) Define the exception class CustomerCardException.

(1 mark)

(b) Implement the Customer class.

(15 marks)

(c) Create the following objects:

(i) Create both drivers and store them in a list, referenced by drivers.

DriverId	Name	Contact
1	John	*98989898*
2	Tom	*88998899*

(ii) Create a customer with name Peter and referenced by cust.

(iii) Create a trip travel card with card number C123 and add the trips for cust. Then create a distance travel card with card number D031 for cust.

Card No	Expiry Date	Trip				
		Trip date	Start Point	Destination	Distance	Driver
C123	*30 Jun 2021*	*30 May 2020, 12:35*	*Jurong Point*	*Parkway Parade*	*20.3*	*2 Tom* 88998899
		30 May 2020, 17:45	*Parkway Parade*	*Tampines Mall*	*11*	*1 John 98989898*
D031	*01Jan 2020*					

(6 marks)

(d) Assume an application has been partially implemented with the menu:

```
Menu
1. Buy a Card
2. Take a trip
3. Top Up
0. Exit
```

Implement the function takeATrip(cust, drivers) for option 2. The function is called using the list drivers and the customer cust created in part Q3(c). The function reads the necessary input data, randomly picks one driver in the list for the trip, and chooses the first card from calling the method getUsableCardNumbers. The function also calls the method travel for the trip.

You must handle the CustomerCardException exception as well as exception caused by input errors. You may assume that user enters the trip date and time in the correct format.

Sample run for option 2 with user input underlined:

```
Enter a choice: 2
Enter trip date in d/m/YYYY hh:mm format: 1/7/2020 8:50
Enter starting point: Jurong Point
Enter destination: Parkway Parade
Enter distance: abc
Distance must be a number

Enter a choice: 2
Enter trip date in d/m/YYYY hh:mm format: 30/6/2020 11:50
Enter starting point: Jurong Point
Enter destination: Parkway Parade
Enter distance: 20.3
No card can be used for trip. Buy a new card or top up a card first

Enter a choice: 2
Enter trip date in d/m/YYYY hh:mm format: 30/6/2020 11:50
Enter starting point: Jurong Point
Enter destination: SUSS
Enter distance: 8
Utilising C123 Balance Detail: Long trips: 0 Short trips: 1
Take trip:  True
```

(13 marks)

Question 4

Develop GUI for an application based on user requirements. The GUI helps customer plan and compute the total cost of trips based on Table Q4.

Calculation based on	Cost
Distance (km)	$1 per km
Type of trip	$10 for trips 12km and under $20 for trips beyond 12km

Table Q4

You do not need to use any of the classes in Q1 to Q3. Refer to the partial code in Figure Q4(e).

Figure Q4(a) shows the initial GUI. When the user clicks on the button

- Add Trip
  If the selected radiobutton is By Trip, the distance entered determines the type of trip (either long or short), and 1 is added to either self._longTrip or self._shortTrip accordingly.
  If the selected radiobutton is By Distance, the distance entered is added to self._distance.
  The GUI shows the values of these variables before and after the increment, as shown in Figure Q4(b).

- Get Total
  The GUI computes the total cost for each category of trips based on Table Q4 and shows the computation, as shown in Figure Q4(c).

- Clear
  The GUI clears the text field for distance and selects the radiobutton By Trip, as shown in Figure Q4(d).

- Reset
  The GUI clears the text field for distance, selects the radiobutton By Trip, clears the scrolltext and sets the variables self._longTrip, self._shortTrip and self._distance to 0 (zero), thus, reverting to the GUI as shown in Figure Q4(a).

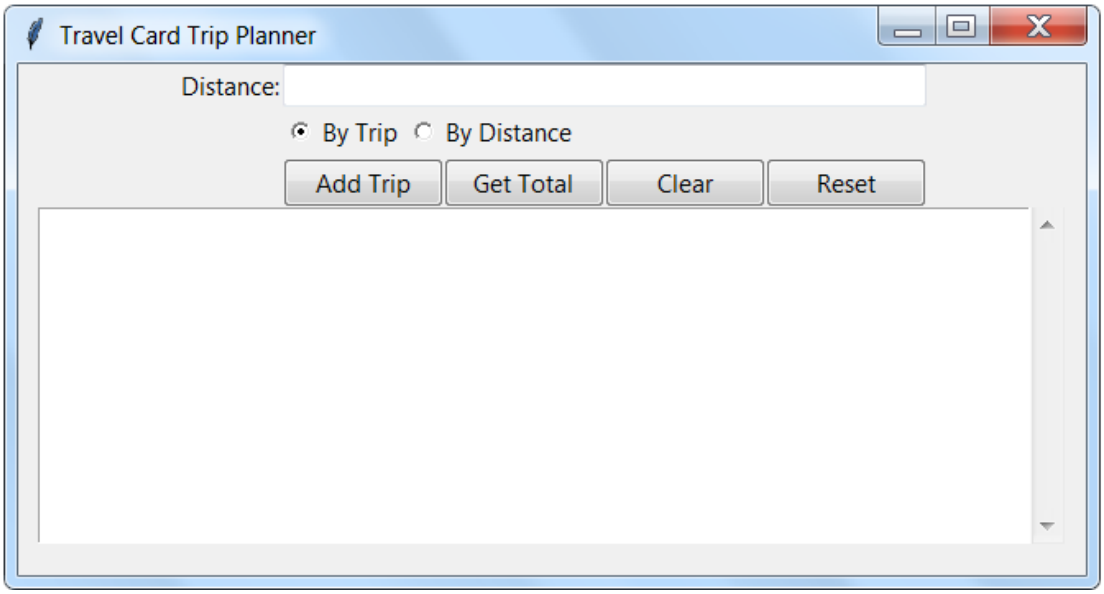

Figure Q4(a)

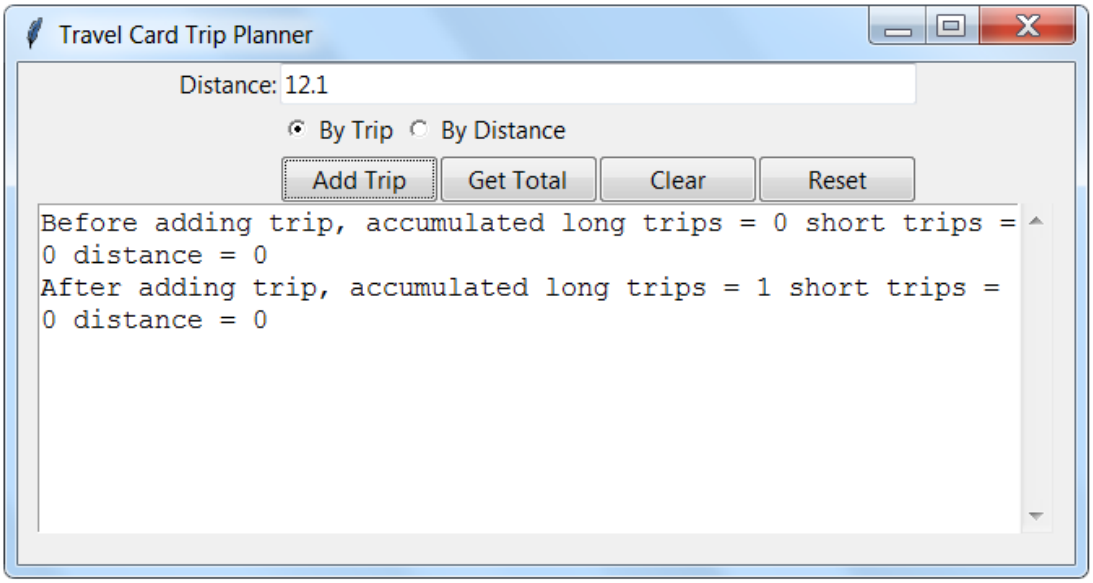

Figure Q4(b)

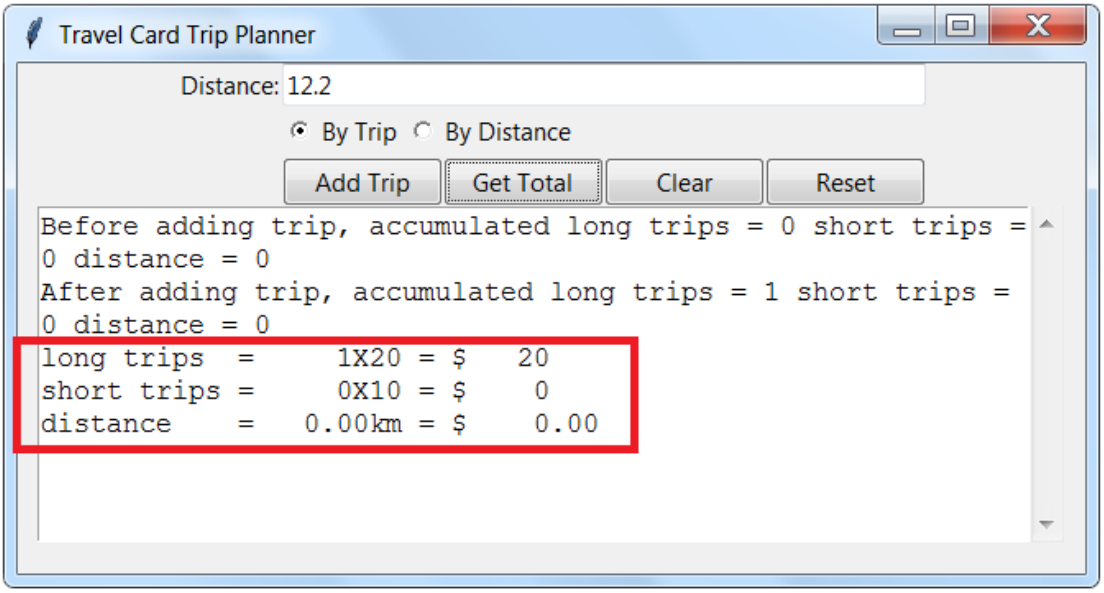

Figure Q4(c)

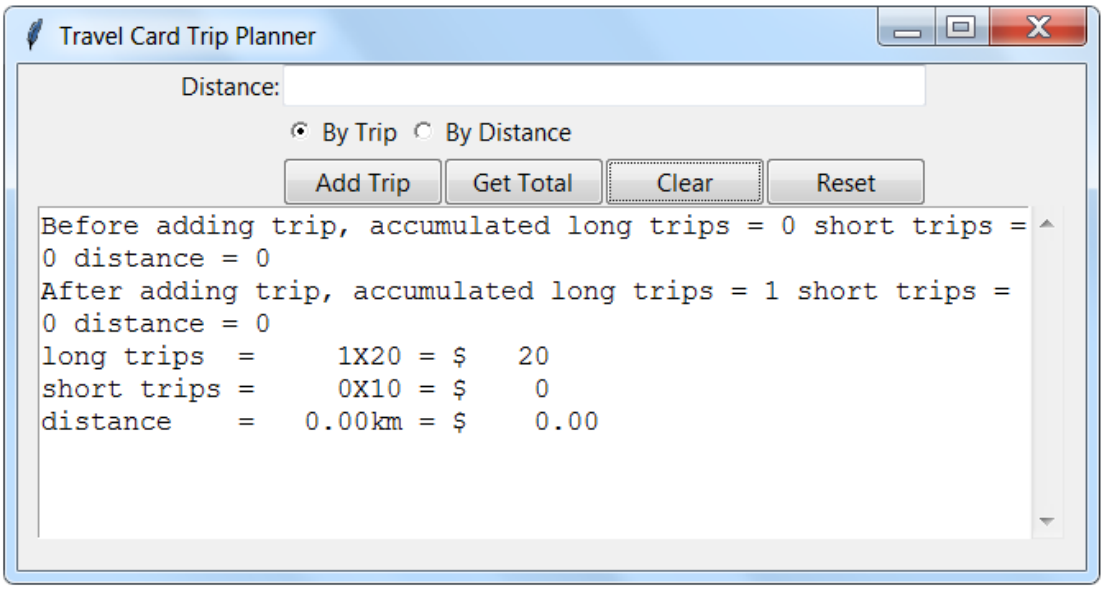

Figure Q4(d)

```
import tkinter as tk
from tkinter import ttk
from tkinter import scrolledtext

class TravelCardGui:

    def __init__(self):
        self._longTrip = 0
        self._shortTrip = 0
        self._distance = 0
        self._win = tk.Tk()
        self._win.title("Travel Card Trip Planner")
        self.create_widgets()
        self._win.geometry('650x310')
        self._win.mainloop()
```

```
    def enableOutput(self):
        self.scrol_stxt.config(state = tk.NORMAL)

    def disableOutput(self):
        self.scrol_stxt.see('end')
        self.scrol_stxt.config(state = tk.DISABLED)

    def clearOutput(self):
        self.enableOutput()
        self.scrol_stxt.delete(1.0,tk.END)
        self.disableOutput()

    def writeToScrollText(self, message):
        self.enableOutput()
        self.scrol_stxt.insert('end', f'{message}'+'\n')
        self.scrol_stxt.see('end')
        self.disableOutput()

    def create_widgets(self):
        dataFrame = ttk.Frame(self._win)
        self._distance_lbl = ttk.Label(dataFrame, text="Distance:")
        self._distanceValue = tk.StringVar()
        self._distance_Ety = ttk.Entry(dataFrame, width=18,
textvariable=self._distanceValue)
        self._distance_lbl.grid(column=0, row=0)
        self._distance_Ety.grid(column=1, row=0, sticky=tk.EW)

        radioFrame = ttk.Frame(dataFrame)
        radioFrame.grid(column=1, row=1, sticky=tk.EW)
        self._cardType = tk.IntVar()
        self._cardType.set(0)
        trip_rdbtn = tk.Radiobutton(radioFrame, text = 'By Trip',
variable=self._cardType, value=0)
        distance_rdbtn = tk.Radiobutton(radioFrame, text = 'By
Distance', variable=self._cardType, value=1)
        trip_rdbtn.grid(column=1, row=0, sticky=tk.W)
        distance_rdbtn.grid(column=2, row=0, sticky=tk.W)

        actionFrame = ttk.Frame(dataFrame)
        actionFrame.grid(column=1, row=3)
        self._travel_btn = ttk.Button(actionFrame, text="Add Trip")
        self._travel_btn.pack(side = tk.LEFT)
        self._total_btn = ttk.Button(actionFrame, text="Get Total")
        self._total_btn.pack(side = tk.LEFT)
        self._clear_btn = ttk.Button(actionFrame, text="Clear")
        self._clear_btn.pack(side = tk.LEFT)
        self._reset_btn = ttk.Button(actionFrame, text="Reset")
        self._reset_btn.pack(side = tk.LEFT)

        dataFrame.pack(side = tk.TOP)
        outputFrame = ttk.Frame(self._win)
        outputFrame.pack(side = tk.TOP)
        self.scrol_stxt = scrolledtext.ScrolledText(outputFrame,
width=60, height=10, wrap=tk.WORD)
        self.scrol_stxt.grid(column=0, row=0, sticky='WE',
columnspan=2)
```

Figure Q4(e)

(a) Show the containment (parent-child) hierarchy of the GUI components.

(6 marks)

(b) Implement event handling so that the GUI can respond to left button clicks on all the buttons.

(14 marks)

----- END OF PAPER -----